Performance Analysis Sparse-Matrix Vector Multiplication using MPI and OpenMP

Chiara Strozzi
Student ID: 243961
Email: chiara.strozzi@studenti.unitn.it
GitHub: https://github.com/chiarastrozzii/PARCO-Computing-2026-243961.git

***Abstract*—This paper illustrates how the performance of Sparse Matrix Vector Multiplication (SpMV) varies using the Message Passing Interface standard for distributed memory parallel computing [MPI] on a multicore platform. SpMV is stored in Compressed Sparse Matrix (CSR) format and distributed across processes using two different decomposition strategies: a 1D row-based partitioning and a 2D block partitioning based on a Cartesian process grid [1]. Performance is evaluated through both strong and weak scaling experiments. Strong scaling exploits real world sparse matrices in Matrix Market format, while weak scaling tests are conducted on synthetic random matrices generated via a Python script. Finally, for the weak scaling evaluation, a hybrid parallel approach combining MPI and OpenMP is also explored to exploit both intra-node and inter-node parallelism [2], [3]. Experiments were performed on the institutional HPC cluster using up to 128 MPI processes. The results show how data decomposition strategy, communication overhead, and process configuration significantly impact the performance of distributed SpMV.**

I. Introduction

SpMV is a core computational kernel in many scientific fields such as machine learning, graph processing and large-scale simulations. Because of high bandwidth requirements, inefficient cache usage and low computational intensity, the efficient computation of the kernel is challenging. In distributed memory systems, the efficiency of SpMV is strongly affected by how the sparse matrix and the input vector are partitioned among processes and by the communication pattern employed between processes [4], [5]. The goal of this paper is to study the improvement in performance by exploiting different decomposition strategies. A baseline implementation based on a 1D cyclic row partitioning is first considered, where each process owns a subset of matrix rows and performs a ghost-entry exchange to obtain the non-local vector elements required for the local SpMV computation. Afterwards, we extend the study to a 2D partitioning scheme, where the matrix is distributed across a Cartesian process grid reducing communication volume. It was proven that by improving the data layout for this problem, it can be possible to reduce SpMV time by 69% [1]. Additionally, a hybrid MPI + OpenMP approach is evaluated to leverage shared-memory parallelism within the single node after a comparison between the 2 separate approaches

II. State of the art

Firstly to further improve the performance of the algorithm we explore the two-dimensional (2D) block partitioning based on a Cartesian process grid. Unlike the one-dimensional (1D) partitioning which is mainly row-based, the 2D partitioning maps the sparse matrix onto a Cartesian process grid, where each process owns a submatrix defined by a contiguous block of rows and columns. This distribution limits the communication volume required during the multiplication by limiting the data exchanges [5], [6]. As a result, the 2D partitioning reduces the communication time and improves load balance, particularly for large and irregular sparse matrices. Furthermore, we also approach a hybrid programming model combining MPI and OpenMP [2] which has become increasingly popular in modern HPC systems. These systems, and the one used to test the different approaches, are based on clusters of multi-core nodes. MPI is used to exploit distributed memory parallelism across nodes, while OpenMP enables shared memory parallelism within each node. An algorithm exploiting both methodologies can reduce the total number of MPI processes, subsequently reducing the communication overhead and memory usage, while still exploiting all available cores through the OpenMP threads. This can offer improved performance for kernels such as SpMV, particularly in weak scaling measurements where the problem size grows with the available computational resources.

III. Contribution and Methodology

A. Sparse Matrix Representation (CSR)

A sparse matrix can be defined as a matrix containing mainly zero entries. Since storing such a matrix in computer memory with all zero values included is inefficient, a compact representation that stores only the non-zero entries is required. One of the most commonly used formats is the Compressed Sparse Row (CSR) format. In CSR, all non-zero values are stored contiguously in an array `val`, ordered row by row. The starting offset of each row in the `val` array is stored in a separate array `row_ptr`, while the array `col_idx` contains the original column index corresponding to each non-zero entry[3].

Algorithm 1 Sparse Matrix–Vector Multiplication (CSR)

```
for i = 0 to n_rows − 1 do
    y[i] ← 0
    for k = row_ptr[i] to row_ptr[i + 1] − 1 do
        j ← col_idx[k]
        y[i] ← y[i] + val[k] × x[j]
    end for
end for=0
```

B. 1D partitioning

In the 1D partitioning approach, the sparse matrix is distributed across MPI processes using a cyclic row-wise scheme, where global row i is assigned to process $i \bmod P$, with P the number of processes. Rank 0 reads the global matrix and assigns each non-zero entry to its owning process based on the row index. The entries are then redistributed using `MPI_Scatterv`, and global row indices are remapped to local indices while column indices remain global. Each process stores its local rows in CSR format and computes the corresponding portion of the result vector.

Since the matrix is partitioned only by rows, local rows may reference vector entries owned by other processes. To address this, a ghost-entry exchange is performed. Each process identifies the unique column indices appearing in its local matrix and separates them into locally owned and remote entries according to the same cyclic distribution. Required vector elements are requested from their owning processes using an all-to-all communication pattern. After the exchange, each process builds a compact local vector containing both owned and ghost entries and remaps the matrix column indices accordingly, enabling the local SpMV computation.

Finally, partial result vectors are gathered on the root process and reordered to reconstruct the global result. This strategy serves as a baseline for comparison with the 2D decomposition.

Algorithm 2 Ghost Entry Exchange for 1D SpMV

Local CSR matrix A_{loc}, locally owned vector entries x_{owned}
Local vector x_{loc} with owned and ghost values
Extract all column indices from A_{loc} Compute unique column set $\mathcal{C}$
Split $\mathcal{C}$ into:

- owned columns $\mathcal{C}_{own}$ where $owner(c) = rank$
- ghost columns $\mathcal{C}_{ghost}$ where $owner(c) \neq rank$

Count how many ghost columns are needed from each process
Exchange ghost column requests using `MPI_Alltoallv`
For each received request:

- retrieve corresponding value from x_{owned}

Send requested values back using `MPI_Alltoallv`
Construct local vector:

$$x_{loc} = [x_{owned}, x_{ghost}]$$

Remap global column indices of A_{loc} to local indices in x_{loc}
return x_{loc}

C. 2D partitioning

To reduce communication overhead, a two-dimensional block partitioning strategy is implemented. MPI processes are arranged in a Cartesian grid of size $p \times q$ using `MPI_Cart_create`, with each process responsible for a contiguous block of rows and columns of the global matrix. For each non-zero entry (i, j), the owning process is determined by its corresponding row and column blocks. The root process distributes the non-zero entries accordingly, and each process stores its local submatrix in CSR format with locally remapped indices.

The input vector is partitioned consistently with the column blocks: each block of the vector is distributed to the corresponding process column and broadcast within the column communicator. As a result, every process holds exactly the vector entries required for its local multiplication.

Unlike the 1D approach, no ghost-entry exchange is required, since both matrix and vector are distributed across two dimensions and all necessary data is available locally after the initial communication. Partial results are reduced across process rows and gathered at the root process to reconstruct the final vector. By avoiding irregular all-to-all communication, the 2D decomposition significantly reduces communication overhead [3].

Algorithm 3 2D Vector Distribution for SpMV

Arrange MPI processes in a $p \times q$ Cartesian grid
Each process (p_r, p_c) owns a matrix block A_{p_r,p_c}
if $p_r = 0$ **then**
    Receive corresponding block of vector x_{p_c} from root
end if
Broadcast x_{p_c} within column communicator
Each process now holds all vector entries needed for A_{p_r,p_c}
Compute local SpMV: $y_{local} = A_{p_r,p_c} \cdot x_{p_c}$
Reduce partial results across row communicator
Gather final result on root process =0

D. Hybrid execution model

A hybrid MPI+OpenMP approach is employed to exploit intra-node parallelism. MPI handles data distribution using both 1D and 2D decompositions, while OpenMP parallelizes the local SpMV within each process across matrix rows. To limit inter-node communication overhead, the number of MPI processes is capped at 16, with additional parallelism obtained by increasing the number of OpenMP threads up to 128.

Hybrid experiments are conducted strictly under weak scaling, where the global problem size grows proportionally with the total number of threads to maintain constant workload per thread. Strong scaling is not considered, as intra-node synchronization and communication overhead would dominate performance. Both 1D and 2D decompositions are evaluated to analyze the interaction between data partitioning and shared-memory parallelism.

IV. Experiments and system description

A. Strong and Weak Scaling

To evaluate the performance and scalability of the SpMV kernel, both strong and weak scaling experiments are conducted. Strong scaling measures how the execution time of a fixed-size problem varies as the number of parallel processes increases, while weak scaling evaluates how the execution time evolves when the problem size grows proportionally with the available parallel resources, maintaining an approximately constant workload per process. Ideally, weak scaling should result in nearly constant execution time.

Both scaling studies are performed using the two partitioning strategies introduced above.

*a) **Strong scaling configuration**:* Strong scaling experiments are carried out using real-world sparse matrices from the Matrix Market collection. For each matrix, the number of MPI processes is increased while keeping the matrix size fixed. Experiments are executed in two modes: `SEQ`, with OpenMP disabled, and `PAR`, where OpenMP is enabled but restricted to a single thread (`OMP_NUM_THREADS` $= 1$), isolating the effects of MPI parallelism. This configuration allows MPI scalability to be evaluated independently while quantifying the overhead introduced by the OpenMP runtime.

*b) **Weak scaling configuration (MPI)**:* For weak scaling with pure MPI, synthetic sparse matrices are generated such that both the global matrix size and the number of non-zero elements increase proportionally with the number of MPI processes, ensuring a constant workload per process. As in the strong scaling experiments, both `SEQ` and `PAR` modes are considered, with the `PAR` mode using a single OpenMP thread.

*c) **Weak scaling configuration (MPI + OpenMP)**:* The hybrid MPI+OpenMP implementation is evaluated exclusively under weak scaling in `PAR` mode. The total level of parallelism is increased by combining MPI processes and OpenMP threads, while the problem size is scaled proportionally to the total number of threads. To limit communication overhead and oversubscription, the number of MPI processes is capped at 16, with additional parallelism obtained through OpenMP threading up to 128 threads. This setup reflects typical execution strategies on modern multicore clusters.

B. Benchmarking Procedure and Hardware

All experiments are performed on a multi-core PBS-managed HPC cluster with 142 CPU nodes (7674 cores), 10 GPU nodes and 65 TB of RAM, using MPI for inter-process communication and OpenMP for shared-memory parallelism within each node. Jobs are launched via `mpirun`, with resources managed by the cluster scheduler.

Benchmarking is automated through scripts that iterate through:

- the input matrices,
- the number of MPI processes,
- the partitioning strategy (1D and 2D),
- the execution mode (`SEQ` and `PAR`).

Strong scaling experiments use fixed-size real-world matrices from the Matrix Market collection, increasing the number of MPI processes from 1 to 128. The selected matrices cover a broad range of sizes and sparsity patterns, including structured and irregular layouts, as well as symmetric and non-symmetric cases. Weak scaling experiments employ synthetic matrices whose size and number of non-zero entries grow proportionally with the parallelism level, ensuring constant work per MPI rank (MPI-only) or per thread (hybrid MPI+OpenMP).

Execution times are collected over multiple runs and automatically aggregated into CSV files for comparative analysis.

C. Performance Metrics

The following metrics are used to evaluate performance and scalability:

- **Execution time:** Average time per SpMV measured over ten timed iterations following an initial warm-up. The maximum execution time across MPI ranks is reported to reflect overall parallel performance. The 90th percentile (P90) is additionally reported to analyze variability across runs.
- **Strong scaling:** Speedup and parallel efficiency computed relative to the single-process baseline.
- **Weak scaling:** Efficiency evaluated by comparing execution time against the corresponding single-process configuration.
- **Communication overhead:** Time spent in data redistribution phases and volume of communicated data per rank, enabling comparison between partitioning strategies.
- **Computational performance:** Achieved floating-point performance (GFLOP/s), derived from the total number of non-zero elements processed per iteration.
- **Memory footprint:** Per-process memory usage, including local CSR storage, ghost entries, and additional data structures introduced by distribution.

V. Results and Discussion

A. Strong Scaling

Strong scaling experiments were conducted using pure MPI with a single thread per process, comparing the 1D and 2D decomposition strategies. Speedup and parallel efficiency were computed with respect to the sequential baseline.

The 1D decomposition exhibits limited scalability: while some speedup is observed at small process counts, performance quickly saturates and efficiency drops as communication overhead from ghost-entry exchanges becomes dominant. Results show that execution times increased 4x from 32 to 128 processes.

In contrast, the 2D decomposition consistently achieves higher speedup and efficiency across all matrices, maintaining better scalability at larger process counts. At large MPI counts, the 2D decomposition was typically 2–4× faster than 1D, with the largest gains observed for irregular matrices where ghost-entry communication dominated as shown in Fig. 1. This behavior results from its balanced data distribution and reduced communication volume, since global all-to-all

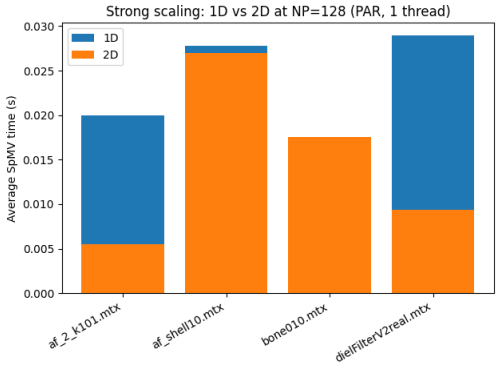

Fig. 1: Strong scaling performance at 128 MPI processes (PAR mode, one thread). The 2D decomposition consistently outperforms 1D due to lower communication overhead.

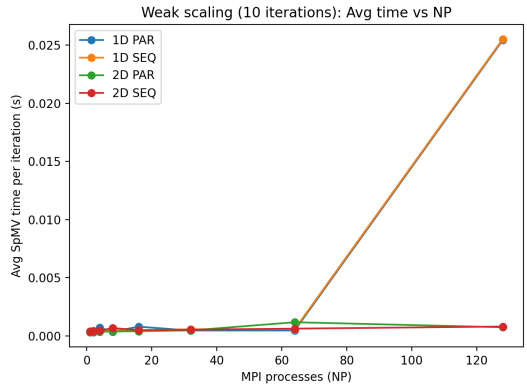

Fig. 2: Weak scaling performance showing average SpMV time per iteration as a function of the number of MPI processes. The 1D decomposition exhibits a sharp increase in execution time at high process counts due to ghost-entry communication overhead.

exchanges are avoided [4], [6]. In addition to runtime and scalability, computational throughput was evaluated in terms of achieved floating-point performance. The 2D decomposition consistently achieved higher GFLOP/s than the 1D strategy, reaching up to 10.4 GFLOP/s compared to 3.4 GFLOP/s for 1D at 128 MPI processes, indicating more effective resource utilization due to reduced communication overhead. These results confirm that 2D partitioning is more suitable for strong scaling of SpMV.

B. Weak Scaling (Pure MPI)

Weak scaling experiments employed synthetic matrices scaled to maintain constant work per process. Ideally, execution time should remain approximately constant as the number of processes increases.

Both decompositions perform similarly at small scales; however, their behavior diverges as parallelism grows. With 1D partitioning, execution time increases rapidly due to the growing cost of ghost-entry communication, leading to declining weak efficiency. Fig. 2 illustrates this rapid growth in execution time beyond 64 processes.

The 2D decomposition shows substantially better weak scaling, with more stable execution times and higher efficiency across process counts. Its localized communication pattern limits overhead growth, making it more effective for large-scale weak scaling.

C. Weak Scaling (Hybrid MPI + OpenMP)

Hybrid experiments were conducted only in parallel mode under weak scaling conditions, combining up to 16 MPI processes with up to 128 OpenMP threads per process. The problem size was scaled with the total number of threads.

For the 1D decomposition, increasing thread-level parallelism yields limited benefits: while local computation accelerates, communication overhead remains dominant, causing weak efficiency to degrade as parallelism increases.

In contrast, the 2D decomposition benefits significantly from hybrid execution. By reducing the number of MPI ranks and exploiting shared-memory parallelism within blocks, communication costs are mitigated and memory locality improves, resulting in more stable execution times and higher efficiency. This highlights that hybrid parallelism is effective primarily when paired with communication-aware decomposition strategies.

D. Additional Experiments

Pure MPI vs. Pure OpenMP (Small Matrices). As an additional experiment, pure OpenMP and pure MPI implementations were compared under strong scaling using smaller matrices. The results show that OpenMP achieves good performance at low core counts due to shared-memory execution and the absence of communication overhead. In contrast, pure MPI introduces non-negligible communication and synchronization costs that limit scalability for small problem sizes. This comparison highlights that shared-memory parallelism is more effective for small workloads, whereas MPI-based parallelism becomes advantageous only as the problem size increases.

VI. Conclusions

This work evaluated the scalability of sparse-matrix vector multiplication using different parallelization strategies and data decompositions. From the analysis of the results of strong scaling with pure MPI, 2D decomposition appears to get higher overall performance and speedup compared to the 1D strategy. The quick degradation of the speedup is caused by the communication overhead which seems to be the greatest limitation of the 1D partitioning. This is further confirmed by the weak scaling experiments where the 2D decomposition maintains a more stable execution time with higher weak efficiency. Finally, hybrid MPI + OpenMP weak scaling experiments highlight that shared-memory parallelism alone is insufficient to overcome inefficient communication patterns. Meaningful benefits are observed when hybrid parallelism is combined with 2D decomposition. Overall, the results confirm that communication patterns play a dominant role in the performance of the SpMV and that 2D partitioning provides the most robust solution.

REFERENCES

[1] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. van der Vorst, *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*, SIAM, Philadelphia, 1993.

[2] G. Schubert, G. Hager, H. Fehske, and G. Wellein, *Parallel Sparse Matrix-Vector Multiplication as a Test Case for Hybrid MPI+OpenMP Programming*, in Proceedings of the IEEE International Symposium on Parallel and Distributed Processing Workshops and PhD Forum (IPDPSW), Anchorage, AK, USA, 2011, pp. 1751–1758, doi: 10.1109/IPDPS.2011.332.

[3] L. Yang, Z. Huang, W. Gao, and Y. Chen, *Hybrid MPI/OpenMP sparse matrix–vector multiplication on many-core architectures*, in Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2021, pp. 492–501.

[4] A. Azad, G. Bhi, T. Scogland, W. Gropp, and L. V. Kale, *A communication-efficient parallel sparse matrix–vector multiplication algorithm*, in Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2015, pp. 825–836.

[5] G. Ballard, J. Demmel, O. Holtz, and O. Schwartz, *Minimizing communication in numerical linear algebra*, SIAM Journal on Matrix Analysis and Applications, vol. 32, no. 3, pp. 866–901, 2011.

[6] A. Buluç and J. R. Gilbert, *Parallel sparse matrix–matrix multiplication and indexing: Implementation and experiments*, SIAM Journal on Scientific Computing, vol. 34, no. 4, pp. C170–C191, 2012.